Projet C++ : Matrice et opérations linéaires

Adrien Evo
Rémi Schmitt
Théo Verdelhan

Février 2026

1 Présentation du sujet

L'objectif de ce projet est de concevoir et implémenter une classe `Matrix` en C++ permettant de manipuler des matrices réelles et d'effectuer les opérations matricielles fondamentales.

La classe doit :

- Stocker les coefficients dans un `std::vector<double>`
- Mémoriser les dimensions (nombre de lignes et colonnes)
- Implémenter les opérations suivantes :
  - Addition matricielle
  - Multiplication matrice × matrice
  - Multiplication matrice × vecteur
  - Transposée
  - Déterminant (pour matrices carrées de petite taille)
- Surcharger les opérateurs :
  - `operator+`
  - `operator*`
  - `operator==`
  - `operator()` pour l'accès aux coefficients
- Gérer les erreurs via :
  - `std::invalid_argument` pour dimensions incompatibles
  - `std::out_of_range` pour indices hors limites

Cette implémentation constitue une base pour le calcul matriciel en algèbre linéaire numérique.

1.1 Enjeux de l'implémentation

1.1.1 L'importance de la contiguïté mémoire : le piège du `std::vector< std::vector<double> >`

Intuitivement, il est tentant de représenter une matrice bidimensionnelle par un "vecteur de vecteurs". Cependant, cette approche est peu performante pour des calculs intensifs.

Un vecteur de vecteurs est en réalité un tableau dynamique de pointeurs, où chaque sous-vecteur (chaque ligne de la matrice) est alloué indépendamment sur le tas (*heap*).

Cette allocation dynamique séparée entraîne une fragmentation de la mémoire : les lignes de la matrice ne se suivent pas physiquement dans la RAM ce qui rend une telle implémentation sous performante.

1.1.2 Optimisation matérielle : Prévention des "Cache Miss"

La mémoire vive (RAM) étant beaucoup plus lente que celle directement sur le CPU (le cache). Lorsqu'il a besoin d'une donnée, le processeur la cherche d'abord dans son cache. S'il la trouve, c'est un Cache Hit. S'il ne la trouve pas, c'est un Cache Miss. Le processeur est alors forcé d'aller chercher la donnée dans la RAM, ce qui est moins performant.

Pour optimiser les transferts, le CPU ne charge jamais une seule variable à la fois depuis la RAM. Il charge des blocs entiers de mémoire contiguë.

Si les données sont contiguës en mémoire, le chargement d'une case de la matrice chargera automatiquement les cases suivantes dans le cache. Avec un vecteur de vecteurs dispersés en mémoire, le CPU doit aller chercher les coefficients dans la RAM ce qui réduit drastiquement la vitesse des algorithmes comme la multiplication de matrices.

1.1.3 La solution : Linéarisation en un vecteur 1D

Pour garantir une occupation mémoire strictement contiguë, l'enjeu principal a été de modéliser cette matrice 2D mathématique à l'aide d'un unique vecteur 1D informatique (`std::vector<double>`).

Ce choix implique une traduction mathématique (linéarisation) des coordonnées. Pour accéder à l'élément situé à la ligne i et à la colonne j d'une matrice de C colonnes, l'indice 1D correspondant est calculé via la formule :

$$\text{Indice} = i \times C + j$$

2 Rappels mathématiques

2.1 Définition générale d'une matrice

Une matrice A de dimension $m \times n$ est un ensemble de $m \times n$ coefficients réels notés :

$$A = (a_{ij})_{1 \leq i \leq m,\ 1 \leq j \leq n}$$

où :
— m représente le nombre de lignes,
— n représente le nombre de colonnes,
— a_{ij} désigne le coefficient situé à la ligne i et à la colonne j.
On peut représenter explicitement une matrice sous la forme :

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix}$$

Les matrices sont utilisées en algèbre linéaire pour représenter :
— des systèmes d'équations linéaires,

— des transformations linéaires,
— des changements de base,
— des données numériques structurées.

Dans ce projet, nous manipulons des matrices à coefficients réels, stockées en mémoire sous forme vectorielle.

2.2 Addition matricielle

Soient A et B deux matrices de dimension $m \times n$.
L'addition est définie par :

$$(A + B)_{ij} = A_{ij} + B_{ij}$$

Condition nécessaire : les matrices doivent avoir les mêmes dimensions.

2.3 Multiplication matrice × matrice

Soit $A \in \mathbb{R}^{m \times n}$ et $B \in \mathbb{R}^{n \times p}$.
Le produit $C = AB$ est une matrice de dimension $m \times p$ définie par :

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$$

Condition nécessaire : le nombre de colonnes de A doit être égal au nombre de lignes de B.

2.4 Multiplication matrice × vecteur

Soit $A \in \mathbb{R}^{m \times n}$ et $x \in \mathbb{R}^n$.

$$(Ax)_i = \sum_{j=1}^{n} A_{ij} x_j$$

Condition : la taille du vecteur doit correspondre au nombre de colonnes de la matrice.

2.5 Transposée

La transposée d'une matrice A est définie par :

$$(A^T)_{ij} = A_{ji}$$

Si A est de dimension $m \times n$, alors A^T est de dimension $n \times m$.

2.6 Déterminant

Le déterminant n'est défini que pour les matrices carrées.

Cas général : développement par cofacteurs

$$\det(A) = \sum_{j=1}^{n} (-1)^{1+j} A_{1j} \det(M_{1j})$$

où M_{1j} est la sous-matrice obtenue en supprimant la première ligne et la j-ième colonne.

Cette méthode est récursive et adaptée uniquement aux matrices de petite taille (complexité factorielle).

3 Choix (non triviaux) concernant l'organisation du code : séparation de l'interface et de l'implémentation

Outre les performances matérielles, un soin particulier a été apporté à l'architecture logicielle de notre classe, en suivant les standards de conception en C++. Cela s'illustre notamment par la gestion des fonctions utilitaires (*helpers*), en particulier celles nécessaires au calcul du déterminant.

3.1 Fonctions libres et principe d'encapsulation

Plutôt que de déclarer les fonctions utilitaires du déterminant comme des méthodes privées de la classe `Matrix`, nous avons fait le choix de les implémenter en tant que fonctions libres (non-membres) directement dans le fichier source (`.cpp`).

Cette approche suit la recommandation de l'expert C++ Scott Meyers : si une fonction peut accomplir sa tâche en se contentant d'appeler l'interface publique d'une classe, elle devrait être implémentée comme une fonction libre plutôt que comme une méthode membre. Cela augmente l'encapsulation de la classe en réduisant le nombre total de fonctions ayant un accès direct à ses données internes privées.

3.2 Isolation via les espaces de noms anonymes

Ces fonctions utilitaires n'ayant pas vocation à être exposées à l'utilisateur de la classe, il était nécessaire de restreindre leur portée. Pour ce faire, nous les avons encapsulées dans un espace de noms anonyme (`namespace { ... }`) au sein du fichier `.cpp`.

En C++, placer une entité dans un espace de noms anonyme lui confère une liaison interne (*internal linkage*). Ces fonctions deviennent alors strictement invisibles et inaccessibles depuis les autres fichiers du projet. Cela permet de ne pas polluer l'espace de noms global et d'éviter les potentielles collisions de noms (contraire à la One Definition Rule) lors de l'étape de linking par le compilateur.

3.3 Optimisation des dépendances de compilation

Ce choix de design a aussi des avantages si notre classe était amenée à être intégrée à un projet plus grand :

Le header file reste concis. Il n'est pas pollué par des détails d'implémentation et agit comme un véritable contrat clair pour l'utilisateur.

Minimisation des recompilations en cascade : Le modèle de compilation du C++ implique que la modification d'un fichier d'en-tête force la recompilation de tous les fichiers sources qui l'incluent. En confinant l'implémentation et les fonctions *helpers* dans le `.cpp`, toute modification (correction de bug, optimisation de l'algorithme du déterminant) nécessite uniquement la recompilation de ce fichier source isolé.

4 Aller plus loin : Perspectives et évolutions

Bien que notre implémentation actuelle réponde aux exigences du sujet, plusieurs pistes d'amélioration peuvent être envisagées pour rendre cette classe matrice plus robuste, performante et générique.

4.1 Redimensionnement dynamique et gestion mémoire

Actuellement, la taille de la matrice est fixe après son instanciation. Permettre l'ajout dynamique de lignes ou de colonnes par l'utilisateur implique de modifier la structure du vecteur 1D sous-jacent.

Si l'ajout d'une ligne à la toute fin de la matrice est une opération relativement peu coûteuse (similaire à des appels successifs à `push_back`), l'insertion d'une colonne, ou d'une ligne au milieu de la matrice, est plus complexe. Elle nécessite de décaler un grand nombre d'éléments en mémoire pour conserver l'ordre de stockage *Row-Major* (ligne par ligne). De plus, ces ajouts peuvent dépasser la capacité allouée du vecteur, forçant une réallocation complète en mémoire. Ces méthodes de redimensionnement doivent donc être pensées avec précaution pour minimiser la dégradation des performances.

4.2 Optimisation algorithmique : Le calcul du déterminant

Dans le cadre de ce projet, le calcul du déterminant repose sur la formule de Laplace (développement par rapport à une ligne ou une colonne). Cette méthode possède une complexité algorithmique factorielle de l'ordre de $\mathcal{O}(n!)$. Si cela ne pose pas de problème majeur pour les petites matrices ciblées par notre sujet, l'algorithme devient très rapidement impraticable lorsque la dimension augmente.

Pour une implémentation à l'échelle, il serait nécessaire de remplacer cette approche par l'algorithme du pivot de Gauss ou la décomposition LU. Ces méthodes permettent de triangulariser la matrice, ramenant la complexité du calcul à $\mathcal{O}(n^3)$ (le déterminant d'une matrice triangulaire se calculant facilement par le produit de ses éléments diagonaux).

4.3 Généricité avec les *Templates*

Notre classe est pour l'instant restreinte au stockage de nombres flottants (`double`). Une évolution naturelle propre au C++ consisterait à la transformer en classe patron (*class template*) via la syntaxe `template <typename T>`, utilisant ainsi un `std::vector<T>`.

Si l'idée est simple, elle soulève des difficultés supplémentaires lors de la surcharge des opérateurs arithmétiques. Il faudrait par exemple gérer proprement les opérations entre matrices de types différents (ex : l'addition d'une `Matrix<int>` avec une `Matrix<double>`). Cela nécessiterait de maîtriser les conversions implicites et d'utiliser des outils modernes comme `std::common_type` pour déduire le type de retour correct sans perte de précision.

4.4 Fonctionnalités mathématiques avancées

Enfin, l'enrichissement de l'interface mathématique ouvrirait la voie à des calculs plus complexes (non linéaires). L'implémentation d'algorithmes de recherche de valeurs propres et de vecteurs propres permettrait d'effectuer la diagonalisation des matrices. Cette propriété est fondamentale : elle offre un accès au calcul rapide des puissances de matrices ($A^k = PD^kP^{-1}$) et, par extension, au calcul de l'exponentielle de matrice (e^A), une opération indispensable dans de nombreux domaines de la physique et de l'ingénierie.